

1º Rec 1º Parcial 2C2024 – Resolución

Teoría

- La utilización de syscalls es necesaria cuando un proceso requiere funciones o recursos que administra el sistema operativo como memoria, semáforos o comunicación entre procesos. Si los procesos pudieran hacer este tipo de operaciones por sí mismos sería un problema grave de seguridad. Al ejecutar una syscall se realiza un cambio de modo ya que su rutina asociada suele contener instrucciones privilegiadas y acceso al espacio de memoria del Kernel.
- Código -----

```
Int contadorGlobal = 0           -> contadorGlobal en Datos  
Int main(int argc, void* argv) {  
    void* variableLocal = malloc(64) -> variableLocal en Stack, la memoria  
    return;                       dinámica pedida en Heap  
}
```
- Por la interrupción de fin de Q el proceso en EXEC pasaría a READY, por la interrupción de fin de E/S un proceso BLOQ pasaría a READY.

En RR sería elegido el proceso que llegue primero a READY, si las interrupciones ocurren en un lapso de tiempo muy corto, dependerá de cuál se atienda primero.

En VRR ejecutaría primero el proceso que estaba bloqueado ya que iría a la cola prioritaria por sobrarle Q a diferencia de quien fue desalojado por fin de Q.
- a. Verdadero, Son las condiciones para que ocurra una condición de carrera y se deba sincronizar, como contra ejemplo, si el acceso no fuera concurrente o ninguno modifique el recurso, no sería necesaria la mutua exclusión.

b. Falso, La implementación de semáforos debe utilizar herencia de prioridades para resolver el conflicto.
- La principal diferencia es en qué momento se emplea cada estrategia. La prevención busca que la petición de recursos se dé de forma tal que no se cumpla alguna de las condiciones necesarias para que ocurra deadlock. La evasión en cambio es algo más flexible con las peticiones pero simula la asignación para verificar que la misma no pueda ocasionar deadlocks en el futuro. Esto requiere un overhead mucho más grande por lo que la prevención es más adecuada en sistemas con menos capacidad ociosa o con una utilización de recursos más predecible.

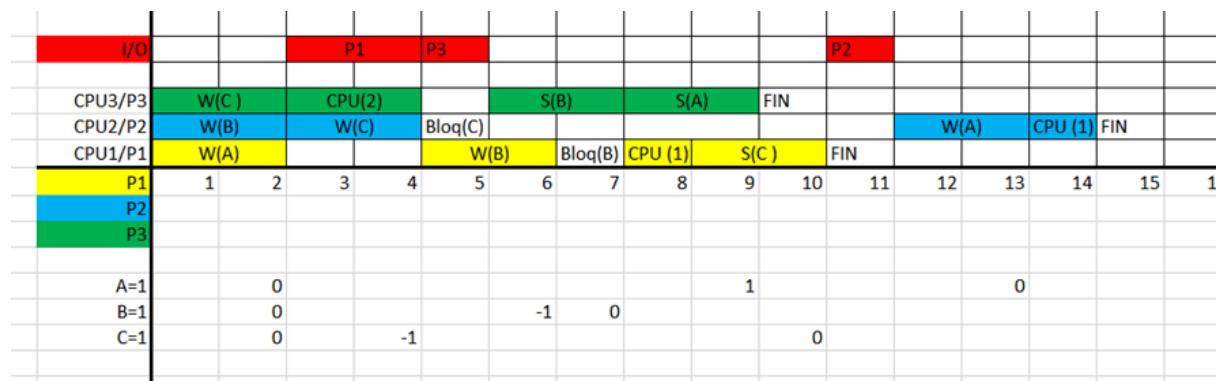
1.

PA	UA1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	UA2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
PB		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
PC		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
PD		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

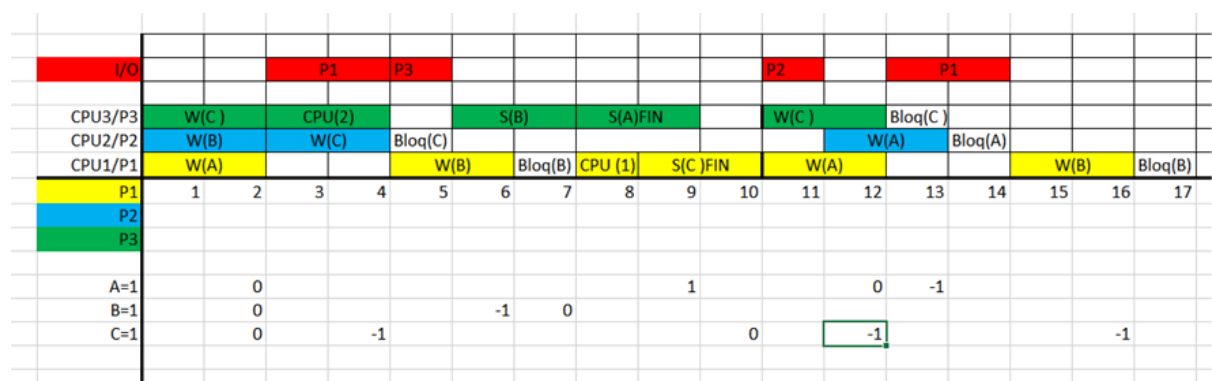
2.

a) El Diagrama muestra que no hay deadlock.

Termina todos los procesos en el orden P3-P1-P2



b) La primera instancia de los procesos P3 Y P1 Termina y vuelven a comenzar. P2 nunca termina su primera instancia de ejecución y luego quedan todos los procesos bloqueados, pero no en deadlock. Dado P1 está bloqueado por el recurso B que lo tienes el mismo asignado. P2 está bloqueado por recurso A que lo tiene P1 y P3 queda bloqueado solicitado el recurso C que lo tiene asignado P2. Esta se da porque la segunda instancia de P1 comienza antes de terminar la primera instancia de P1



3.

Streamer (1 instancia)	Conversor (100 instancias)	Suscriptor (M instancias)
<pre> while(1) { video = editarVideo(); wait(límiteVideos) wait(mutexVideos) subir(video, colaDeVideos); signal(mutexVideos) signal(contVideos) } </pre>	<pre> while(1) { wait(contVideos) wait(mutexVideos) video = obtener(colaDeVideos); signal(mutexVideos) signal(límiteVideos) wait(videosConcurrentes) procesar(video); signal(videosConcurrentes) for(idSus=0; idSus<M; idSus++) { wait(mutexNotif[idSus]) enviar(notif[idSus]); signal(mutexNotif[idSus]) signal(contNotif[idSus]) } } </pre>	<pre> while(1) { wait(contNotif[idSus]) wait(mutexNotif[idSus]) video = abrir(notif[getId()]); signal(mutexNotif[idSus]) disfrutar(video); wait(suscripConcurrentes) renovarSuscripción(); signal(suscripConcurrentes) } </pre>

límiteVideos = 10

contVideos = 0

mutexVideos = 1

videosConcurrentes = 5

suscripConcurrentes = 20

contNotif[M] = {0, 0, 0, 0,}

mutexNotif[M] = {1, 1, 1, 1,}